

Ch_14 체인(Chains)

문서 기반 RAG Chain 구조

RAG 7단계 (문서 기반 기준)

1. 문서 로드 → 문서 불러오기
2. 문서 분할 → 문서를 작은 조각(Chunk)로 나눔
3. 임베딩 생성 → 각 문서 조각을 숫자 벡터로 변환 / 의미 기반 검색을 가능하게 함
4. 벡터 DB 저장 → 변환된 벡터를 DB에 저장 / 나중에 유사도 검색 가능
5. 질문 입력 → 사용자가 질문을 입력함
6. 검색 (Retriever) → 질문을 임베딩으로 바꾼 후, DB에서 관련 문서 조각들 찾기
7. LLM 답변 생성 → 검색된 문서(context) + 질문 합쳐서 LLM이 최종 답변 생성

질문 → Retriever → 문서 검색

↓

(context + question)

↓

LLM

↓

답변

체인 생성

```
chain = (  
    {"context": retriever, "question": RunnablePassthrough  
    ()}  
    | prompt  
    | llm  
    | StrOutputParser()  
)
```

"context": retriever, "question": RunnablePassthrough()

retriever → 질문을 받아서 관련 문서를 찾아 context로 넣기

RunnablePassthrough() → 질문을 그대로 question으로 전달

입력:

```
"삼성전자가 자체 개발한 AI 의 이름은?"
```

자동으로 변환:

```
{  
  "context": (검색된 문서들),  
  "question": "삼성전자가 자체 개발한 AI 의 이름은?"  
}
```

| prompt

위에서 만든 `context+question`을 Prompt Template에 넣음

```
다음 정보를 바탕으로 답하세요.
```

```
context: (검색 결과)
```

```
question: 삼성전자가 자체 개발한 AI 의 이름은?
```

| llm

프롬프트를 LLM에 보내고 답변 생성

| StrOutputParser()

LLM의 결과를 문자열 형태로 정리해서 출력

I. 문서 요약

i) Stuff

: 문서 목록을 모두 가져와서 프롬프트에 삽입한 다음, 그 프롬프트를 LLM에 전달함
한국어로 요약 작성하라는 문구가 추가된 prompt

```
from langchain import hub

prompt = hub.pull("teddynote/summary-stuff-documents-korean")
prompt.pretty_print()
```

ii) Map-Reduce

: 긴 문서를 효율적으로 요약하는 기법

- 문서를 작은 chunk로 나눈 "map" 단계
 - chunk를 병렬로 요약

batch()를 호출하여 각 문서에 대한 요약본을 생성

```
# 문서에 대한 주요내용 추출
doc_summaries = map_chain.batch(docs)
```

- 각 chunk의 요약을 결합하는 "reduce" 단계
 - 요약된 chunk들을 하나의 최종 요약으로 통합

```
# reduce prompt 다운로드
reduce_prompt = hub.pull("teddynote/reduce-prompt")

# 프롬프트 출력
reduce_prompt.pretty_print()
```

Reduce Chain 생성

```
# reduce chain 생성
reduce_chain = reduce_prompt | llm | StrOutputParser()
```

Reduce Chain을 사용하여 스트리밍 출력 예시

```
from langchain_teddynote.messages import stream_response

answer = reduce_chain.stream({"doc_summaries": doc_summaries, "language": "Korean"})
stream_response(answer)
```

iii) Map-Refine

: Map-Reduce와 유사하지만 약간의 차이!

1. Map 단계: 문서를 여러 개의 작은 chunk로 나누고, 각 chunk에 대해 개별적으로 요약 생성함.
2. Refine 단계: 생성된 요약들을 순차적으로 처리하며 최종 요약을 점진적으로 개선함. 각 단계에서 이전 요약과 새로운 chunk의 정보를 결합하여 요약을 갱신함.
3. 반복 과정: 모든 chunk가 처리될 때까지 refine 단계를 반복함.
4. 최종 요약: 마지막 chunk까지 처리한 후 얻은 요약이 최종 결과.

장점

- 문서의 순서를 유지하면서 점진적으로 요약을 개선할 수 있다는 것.
 - 이는 특히 문서의 맥락이 중요한 경우에 유용할 수 있음.
- 그러나 이 방식은 map-reduce에 비해 순차적으로 처리되기 때문에 병렬화가 어려워 대규모 문서 처리 시 시간이 더 오래 걸릴 수 있음.

iv) Chain of Density

: GPT-4를 사용한 요약 생성을 개선하기 위해 개발된 기법

1. 점진적 개선: CoD는 초기에 개체가 적은 간단한 요약을 생성한 후, 단계적으로 중요한 개체들을 추가하며 요약을 개선함. → 요약의 길이는 유지되면서 정보 밀도가 증가

2. 정보 밀도와 가독성의 균형: 요약의 정보 밀도를 조절하여 정보성과 가독성 사이의 최적 균형점을 찾음. → 사람들은 GPT-4 요약보다 더 밀도 있지만 사람이 작성한 요약만큼 밀도가 높지 않은 CoD 요약을 선호함.
3. 추상화와 정보 융합 개선: 더 추상적이고 정보 융합이 뛰어나며, 원문에 앞부분에 치우치는 경향(lead bias)이 덜함. → 요약의 전반적인 품질과 가독성을 향상시키는 데 기여함.

v) Clustering-Map-Refine

: 긴 문서의 요약에 대한 것!

1. map-reduce나 map-refine 방식은 모두 시간이 오래 걸리고, 비용이 많이 듦.

⇒ 문서를 몇 개(N 개)의 클러스터로 나눈 뒤, 가장 중심축에서 가까운 문서를 클러스터의 대표 문서로 인지하고, 이를 map-reduce(혹은 map-refine) 방식으로 요약하는 방식을 제안!

II. SQL

`create_sql_query_chain` 을 활용하여 생성한 체인으로 SQL 쿼리를 생성 후 실행!

1. SQL Database 정보를 불러옴

```
from langchain_openai import ChatOpenAI
from langchain.chains import create_sql_query_chain
from langchain_community.utilities import SQLDatabase

# SQLite 데이터베이스에 연결합니다.
db = SQLDatabase.from_uri("sqlite:///data/finance.db")

# 데이터베이스의을 출력합니다.
print(db.dialect)

# 사용 가능한 테이블 이름들을 출력합니다.
print(db.get_usable_table_names())
```

2. LLM 객체를 생성하고 LLM과 DB를 매개변수로 입력하여 chain을 생성함.
3. chain을 실행하면 DB 기반으로 쿼리를 생성함.

```
# chain 을 실행하고 결과를 출력합니다.
generated_sql_query = chain.invoke({"question": "고객의 이름을 나열하세요"})

# 생성된 쿼리를 출력합니다.
print(generated_sql_query.__repr__())
```

Agent

: Agent를 활용하여 SQL 쿼리를 생성하고 실행 결과를 답변으로 출력이 가능함

```
from langchain_openai import ChatOpenAI
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import create_sql_agent

# model 은 gpt-3.5-turbo 를 지정
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# SQLite 데이터베이스에 연결합니다.
db = SQLDatabase.from_uri("sqlite:///data/finance.db")

# Agent 생성
agent_executor = create_sql_agent(llm, db=db, agent_type="openai-tools", verbose=True)

# 실행 결과 확인
agent_executor.invoke(
    {"input": "테디와 셸리의 transaction 의 합계를 구하고 비교하세요"}
)
```

III. 구조화된 출력 체인 (with_structured_output)

특정 주제에 대한 4지선다형 퀴즈를 생성하는 과정을 구현!

- `Quiz` 클래스는 퀴즈의 질문, 난이도, 그리고 네 개의 선택지를 정의함.
- `ChatOpenAI` 인스턴스는 GPT-4o 모델을 사용하여 자연어 처리를 수행하고, `ChatPromptTemplate` 는 퀴즈 생성을 위한 대화형 프롬프트를 정의합니다.

```
from langchain.chains.openai_functions import create_structured_output_runnable
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.pydantic_v1 import BaseModel, Field
from typing import List

class Quiz(BaseModel):
    """4지선다형 퀴즈의 정보를 추출합니다"""

    question: str = Field(..., description="퀴즈의 질문")
    level: str = Field(
        ..., description="퀴즈의 난이도를 나타냅니다. (쉬움, 보통, 어려움)"
    )
    options: List[str] = Field(..., description="퀴즈의 4개의 선택지입니다.")

llm = ChatOpenAI(model="gpt-4o", temperature=0.1)
prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You're a world-famous quizzier and generates quizzes in structured formats.",
        )
    ]
)
```

```

    ),
    (
        "human",
        "TOPIC 에 제시된 내용과 관련한 4지선다형 퀴즈를 출제해
주세요. 만약, 실제 출제된 기출문제가 있다면 비슷한 문제를 만들어 출제
하세요."
        "단, 문제에 TOPIC 에 대한 내용이나 정보는 포함하지 마
세요. \nTOPIC:\n{topic}",
    ),
    ("human", "Tip: Make sure to answer in the correct
format"),
]
)

```

```

# 구조화된 출력을 위한 모델 생성
llm_with_structured_output = llm.with_structured_output(Quiz)

```

```

# 퀴즈 생성 체인 생성
chain = prompt | llm_with_structured_output

```

```

# 퀴즈 생성을 요청합니다.
generated_quiz = chain.invoke({"topic": "ADSP(데이터 분석 준
전문가) 자격 시험"})

```

생성된 퀴즈 출력:

```

# 생성된 퀴즈 출력
print(f"{generated_quiz.question} (난이도: {generated_quiz.level})\n")
for i, opt in enumerate(generated_quiz.options):
    print(f"{i+1}) {opt}")

```

다음 중 데이터 분석의 과정에서 가장 먼저 수행해야 하는 단계는 무엇인가요? (난이도: 보통)

1) 데이터 수집

- 2) 데이터 전처리
- 3) 문제 정의
- 4) 모델 평가